

Abrir el capó

Hoja de recetas. Una hora, en parejas, con un SGBD real (SQLite) y una base de datos de la plataforma de streaming del curso.

LA REGLA DEL TALLER Hoy no aprendemos SQL. Vais a ejecutar órdenes que todavía no entendéis del todo, como quien da una vuelta en coche antes de sacarse el carnet. Cada orden demuestra una promesa de la clase de teoría. El idioma lo aprenderemos con calma más adelante: por ahora, copiad, ejecutad y mirad.

ANTES DE EMPEZAR

- Necesitáis el programa sqlite3 (la consola). En macOS ya viene instalado; en Linux, `sudo apt install sqlite3`; en Windows, descargad `sqlite-tools` de sqlite.org y descomprimidlo en una carpeta.
- Una carpeta de trabajo con el fichero `construye_plataforma.sql` (lo tenéis en el campus virtual).
- Dos ventanas de terminal abiertas en esa carpeta. La segunda solo se usa en el bloque 3.
- Convención: lo que empieza por punto (`.tables`) es una orden para el programa sqlite3; lo que termina en punto y coma (`SELECT ...;`) es una orden para la base de datos. Las de la base de datos no se ejecutan hasta el punto y coma.

Bloque 0 · Construir la base de datos 3 min

Abrid sqlite3 sobre un fichero nuevo y leed el guion de construcción:

```
sqlite3 plataforma.db
.read construye_plataforma.sql
```

Tarda entre 5 y 30 segundos según el ordenador. Acaba de crear tres tablas y de generar **cuatro millones** de reproducciones inventadas. Activad ahora dos ajustes para que las respuestas se lean bien (hay que repetirlos cada vez que abráis sqlite3):

```
.mode column
.headers on
```

PROMESA QUE DEMUESTRA Definir una base de datos y cargar datos son las dos primeras funciones de un SGBD. Cuatro millones de filas en segundos: la escala del ejercicio de Fermi ya no es teórica.

Bloque 1 · Esquema, estado y el catálogo 10 min

¿Qué tablas hay? ¿Cómo es cada una (el esquema)? ¿Qué contiene ahora mismo (el estado)?

```
.tables
.schema USUARIO
SELECT * FROM USUARIO;
.schema CANCION
SELECT * FROM CANCION;
```

Fijaos: `.schema` os enseña el **plano** (nombres, tipos, reglas) y `SELECT` os enseña los **muebles** (las seis filas de hoy). Jon no tiene fecha de nacimiento: guardad esa observación para más adelante.

La tabla grande no la miréis entera. Preguntad solo cuántas filas tiene y mirad cinco:

```
SELECT COUNT(*) FROM ESCUCHA;
SELECT * FROM ESCUCHA LIMIT 5;
```

Y ahora la receta estrella. ¿Dónde guarda el SGBD el esquema? **Dentro de la propia base de datos**, en una tabla llamada `sqlite_master`:

```
.mode list
SELECT name, sql FROM sqlite_master;
.mode column
```

PROMESA QUE DEMUESTRA El catálogo: una base de datos que se describe a sí misma. La columna `sql` contiene, literalmente, la orden que creó cada tabla. Metadatos: datos sobre los datos.

PREGUNTA PARA LA PAREJA ESCUCHA registra escuchas de 50.000 usuarios, pero en `USUARIO` solo hay seis. ¿Debería el SGBD quejarse? ¿Qué regla le falta por conocer? (Apuntad la respuesta: la veremos dentro de dos semanas y tiene nombre.)

OJO No borréis nada de `sqlite_master`. ¿Qué creéis que pasaría?

Bloque 2 · El SGBD conoce el mini-mundo 10 min

Intentad guardar cosas que no deberían poder guardarse. Leed con atención el mensaje de error de cada una: lo escribe el SGBD, no vuestro programa.

```
INSERT INTO CANCION (titulo, artista, duracion, anio)
VALUES ('Prueba', 'Nadie', -3, 2020);

INSERT INTO USUARIO (nombre, email)
VALUES ('Otra María', 'maria@mail.es');

INSERT INTO USUARIO (nombre, email)
VALUES ('Sin arroba', 'hola.mail.es');
```

Tres rechazos: una duración negativa, un email repetido y un email sin arroba. Volved a mirar `.schema` `USUARIO` y localizad la regla que ha disparado cada mensaje (`CHECK`, `UNIQUE`).

Ahora una inserción legal, y comprobad que el estado ha cambiado pero el esquema no:

```
INSERT INTO USUARIO (nombre, email, fecha_nacimiento)
VALUES ('Vera', 'vera@mail.es', '2004-09-12');
SELECT * FROM USUARIO;
.schema USUARIO
```

PROMESA QUE DEMUESTRA Las restricciones viven en la base de datos, no en el programa. Es el fallo 2 del fichero (nadie vigila las reglas), resuelto: da igual desde qué aplicación llegue el dato, el SGBD lo juzga con el esquema.

Una última prueba, que os va a incomodar:

```
INSERT INTO CANCION (titulo, artista, duracion, anio)
VALUES ('Under Pressure', 'Queen, David Bowie', 244, 1981);
SELECT * FROM CANCION;
```

PREGUNTA PARA LA PAREJA La acepta. Dos artistas en una celda, y el SGBD no ha dicho nada. ¿Por qué esta sí y la duración -3 no? ¿Qué le faltaba por saber? Guardad la incomodidad: es la primera pregunta del próximo tema.

Dejadlo limpio: `DELETE FROM CANCION WHERE titulo = 'Under Pressure';`

Bloque 3 · Dos terminales, una fila 15 min

Abrid la **segunda terminal** en la misma carpeta y entrad en la misma base de datos: `sqlite3` `plataforma.db` (con `.mode column` y `.headers on`). Llamaremos **A** a la primera y **B** a la segunda. Seguid el orden exacto; un miembro de la pareja escribe en A y el otro en B.

EN A · empezad una transacción y cambiad el email de María, SIN confirmar

```
BEGIN;
UPDATE USUARIO SET email = 'maria.nueva@mail.es' WHERE id_usuario = 2;
SELECT nombre, email FROM USUARIO WHERE id_usuario = 2;
```

A ve el email nuevo. La transacción está abierta: el cambio existe, pero no está confirmado.

EN B · mirad a María

```
SELECT nombre, email FROM USUARIO WHERE id_usuario = 2;
```

B sigue viendo maria@mail.es. Un cambio a medias no se ve desde fuera.

PROMESA QUE DEMUESTRA Aislamiento (la I de ACID): nadie ve las transacciones de otros hasta que terminan. Es la última entrada del concierto de la clase de teoría.

EN B • intentad escribir vosotros también

```
UPDATE USUARIO SET nombre = 'Leonardo' WHERE id_usuario = 5;
```

Respuesta: database is locked. Mientras A tiene una escritura a medias, nadie más escribe. Dos escritores no se pisan: fallo 4 del fichero, resuelto.

EN A • morid sin confirmar

Cerrad la ventana de la terminal A de golpe (la X de la ventana), sin escribir COMMIT. Si no podéis cerrarla, pulsad Ctrl+C tres veces. El proceso ha muerto con la transacción abierta: es "la luz que se va" de la clase de teoría.

EN B • ¿qué ha quedado?

```
SELECT nombre, email FROM USUARIO WHERE id_usuario = 2;
UPDATE USUARIO SET nombre = 'Leonardo' WHERE id_usuario = 5;
SELECT nombre FROM USUARIO WHERE id_usuario = 5;
```

María sigue con maria@mail.es: la operación a medias **no ha dejado rastro**. Y ahora B sí puede escribir (Leo ya es Leonardo).

PROMESA QUE DEMUESTRA Atomicidad (la A): una transacción que no termina es como si nunca hubiera empezado. El SGBD lo consigue con un diario en disco (el journal) que le permite deshacer lo que quedó a medias. Fallo 5 del fichero, resuelto.

REPETID, AHORA CON COMMIT

Volved a abrir A (sqlite3 plataforma.db) y repetid el cambio de email, esta vez confirmándolo:

```
BEGIN;
UPDATE USUARIO SET email = 'maria.nueva@mail.es' WHERE id_usuario = 2;
COMMIT;
```

Mirad desde B: ahora sí ve maria.nueva@mail.es. Cerrad A de golpe otra vez y volved a mirar desde B: el cambio sigue ahí.

PROMESA QUE DEMUESTRA Durabilidad (la D): lo confirmado sobrevive a cualquier caída posterior. Si el sistema dijo "comprada", está comprada.

PREGUNTA PARA LA PAREJA ¿Y la C de Consistencia? Pensad qué habría pasado si, dentro de la transacción de A, hubierais intentado poner a María el email de Leo. ¿En qué momento lo rechazaría el SGBD? Probadlo si os sobra tiempo.

Dejadlo limpio desde B: UPDATE USUARIO SET nombre = 'Leo' WHERE id_usuario = 5;

Bloque 4 • Independencia física: el índice 15 min

Seguid en una sola terminal. Activad el cronómetro: a partir de ahora sqlite3 os dirá cuánto tarda cada orden (Run Time: real ...).

```
.timer on
```

Dos preguntas sobre la tabla grande. Anotad el tiempo real de cada una.

```
-- Pregunta A: ¿cuántas veces ha escuchado algo María?
SELECT COUNT(*) FROM ESCUCHA WHERE id_usuario = 2;

-- Pregunta B: ¿cuántas escuchas tiene cada uno de nuestros usuarios?
SELECT u.nombre,
       (SELECT COUNT(*) FROM ESCUCHA e WHERE e.id_usuario = u.id_usuario) AS
escuchas
FROM USUARIO u;
```

La pregunta B tarda entre uno y varios segundos: para cada usuario, el SGBD recorre los cuatro millones de filas enteros. Pedidle que os cuente su plan:

```
EXPLAIN QUERY PLAN SELECT COUNT(*) FROM ESCUCHA WHERE id_usuario = 2;
```

Responde SCAN ESCUCHA: "recorro la tabla entera". Ahora, como haría un administrador, construid una estructura de búsqueda sobre la columna del usuario. Tarda unos segundos: está ordenando cuatro millones de valores.

```
CREATE INDEX idx_escucha_usuario ON ESCUCHA (id_usuario);
```

Y repetid **exactamente las mismas dos preguntas** (flecha arriba en el teclado las recupera) y el EXPLAIN QUERY PLAN.

PREGUNTA PARA LA PAREJA ¿Cuánto tardan ahora? ¿Qué dice el plan? Y la pregunta importante: ¿qué habéis cambiado en la consulta? (Nada.)

PROMESA QUE DEMUESTRA Independencia física de datos: cómo se guardan y se buscan los datos ha cambiado por completo, y la pregunta es la misma letra por letra. Y de propina: nadie le ha dicho al SGBD que use el índice — lo ha decidido él. Con pocas operaciones, bien definidas, el SGBD puede optimizarlas por vosotros. Fallos 3 y 6 del fichero, resueltos.

Para ver que el índice es una estructura real que ocupa sitio, borrarlo y volved a preguntar: DROP INDEX idx_escucha_usuario; — los segundos vuelven.

Balance

Cada bloque de hoy encaja en una casilla de la clase de teoría:

Bloque	Lo que has hecho	Promesa de la teoría	Fallo del fichero que tacha
0	Construir la BD desde un guion	El SGBD define y carga: 4 millones de filas en segundos	—
1	.schema, SELECT, sqlite_master	Esquema ≠ estado · el catálogo (autodescripción)	—
2	Inserciones rechazadas	Restricciones: el SGBD conoce el mini-mundo	2 (nadie vigila las reglas)
3	Dos terminales; matar sin COMMIT	Transacciones ACID: aislamiento, atomicidad, durabilidad	4 y 5 (conurrencia, a medias)
4	Índice + .timer + EXPLAIN	Independencia física · optimización automática	3 y 6 (lento, formato)

Hoy habéis tocado la **estructura** (tablas), las **operaciones** (consultas y cambios) y las **restricciones** (los rechazos). Ahora toca entender cada cosa, y empezamos por la estructura.

PARA EL LUNES La tabla CANCION tiene un id_cancion. ¿Por qué? ¿No bastaba con el título para identificar una canción? Pensadlo para la próxima clase — y pensad también en 'Queen, David Bowie'.